

1.

C++ code	Result
<pre> #include<iostream> #include<cstring> using namespace std; #pragma warning(disable:4996) class String { private: char* str; public: String(const char* s) { str = new char[strlen(s) + 1]; strcpy(str, s); cout << "构造了一个String对象" << endl; } String() { str = new char[2]; strcpy(str, "\0"); cout << "构造了一个String对象" << endl; } String(const String& STR) { str = new char[strlen(STR.str) + 1]; strcpy(str, STR.str); cout << "复制构造了一个String对象" << endl; } ~String() { delete[]str; cout << "析构了一个 String对象" << endl; } void Set(const char* s) { delete[]this->str; this->str = new char[strlen(s) + 1]; strcpy(str, s); } void Print() { cout << str << endl; } bool operator == (const String s2) { return !strcmp(str, s2.str); } bool operator > (const String s2) { if (strcmp(str, s2.str) == 1) return true; else return false; } bool operator < (const String s2) { if (strcmp(str, s2.str) == -1) return true; else return false; } bool operator != (const String s2) { return !(*this == s2); } bool operator >= (const String s2) { return !(*this < s2); } bool operator <= (const String s2) { return !(*this > s2); } String& operator = (const String s2) { if (this != &s2) { delete[] str; this->str = new char[strlen(s2.str) + 1]; strcpy(str, s2.str); </pre>	<pre> 构造了一个String对象 构造了一个String对象 s1 = abce s2 = abde 复制构造了一个String对象 复制构造了一个String对象 复制构造了一个String对象 析构了一个String对象 复制构造了一个String对象 析构了一个String对象 复制构造了一个String对象 析构了一个String对象 s1 < s2 复制构造了一个String对象 复制构造了一个String对象 析构了一个String对象 析构了一个String对象 复制构造了一个String对象 复制构造了一个String对象 析构了一个String对象 析构了一个String对象 s1 != s2 复制构造了一个String对象 复制构造了一个String对象 析构了一个String对象 析构了一个String对象 s1 <= s2 析构了一个String对象 析构了一个String对象 s1 = fghj s2 = fejk 复制构造了一个String对象 复制构造了一个String对象 复制构造了一个String对象 析构了一个String对象 s1 > s2 复制构造了一个String对象 析构了一个String对象 复制构造了一个String对象 析构了一个String对象 复制构造了一个String对象 复制构造了一个String对象 析构了一个String对象 析构了一个String对象 s1 >= s2 复制构造了一个String对象 复制构造了一个String对象 析构了一个String对象 析构了一个String对象 s1 != s2 复制构造了一个String对象 复制构造了一个String对象 析构了一个String对象 析构了一个String对象 析构了一个String对象 析构了一个String对象 复制构造了一个String对象 析构了一个String对象 s1 = pubg s2 = pubg 复制构造了一个String对象 </pre>

```

    }
    return *this;
}
};

void cmp(String s1, const String s2) {
    if (s1 > s2) cout << "s1 > s2" << endl;
    if (s1 == s2) cout << "s1 == s2" << endl;
    if (s1 < s2) cout << "s1 < s2" << endl;
    if (s1 >= s2) cout << "s1 >= s2" << endl;
    if (s1 != s2) cout << "s1 != s2" << endl;
    if (s1 <= s2) cout << "s1 <= s2" << endl;
}

int main() {
    String s1("abce"), s2("abde");
    cout << "s1 = "; s1.Print();
    cout << "s2 = "; s2.Print();
    cmp(s1, s2);
    s1.Set("fghj"); s2.Set("fejk");
    cout << "s1 = "; s1.Print();
    cout << "s2 = "; s2.Print();
    cmp(s1, s2);
    s1.Set("pubg"); s2 = s1;
    cout << "s1 = "; s1.Print();
    cout << "s2 = "; s2.Print();
    cmp(s1, s2);
    return 0;
}

```

复制构造了一个 String 对象
 析构了一个 String 对象
 复制构造了一个 String 对象
 析构了一个 String 对象
 s1 == s2
 复制构造了一个 String 对象
 析构了一个 String 对象
 复制构造了一个 String 对象
 复制构造了一个 String 对象
 析构了一个 String 对象
 析构了一个 String 对象
 s1 >= s2
 复制构造了一个 String 对象
 复制构造了一个 String 对象
 析构了一个 String 对象
 析构了一个 String 对象
 复制构造了一个 String 对象
 复制构造了一个 String 对象
 析构了一个 String 对象
 析构了一个 String 对象
 s1 <= s2
 析构了一个 String 对象
 析构了一个 String 对象
 析构了一个 String 对象
 析构了一个 String 对象

C++ code	Result
<pre> #include<iostream> #include<cstring> using namespace std; #pragma warning(disable:4996) class String { private: char* str; public: String(const char* s) { str = new char[strlen(s) + 1]; strcpy(str, s); cout << "构造了一个String对象" << endl; } String() { str = new char[2]; strcpy(str, "\0"); cout << "构造了一个String对象" << endl; } String(const String& STR) { str = new char[strlen(STR.str) + 1]; strcpy(str, STR.str); cout << "复制构造了一个String对象" << endl; } ~String() { delete[]str; cout << "析构了一个 String对象" << endl; } void Set(const char* s) { delete[]this->str; this->str = new char[strlen(s) + 1]; strcpy(str, s); } void Print() { cout << str << endl; } friend bool operator == (const String s1, const String s2) { return !strcmp(s1.str, s2.str); } friend bool operator > (const String s1,const String s2) { if (strcmp(s1.str, s2.str) == 1) return true; else return false; } friend bool operator < (const String s1, const String s2) { if (strcmp(s1.str, s2.str) == -1) return true; else return false; } friend bool operator != (const String s1, const String s2) { return !(s1 == s2); } friend bool operator >= (const String s1, const String s2) { return !(s1 < s2); } friend bool operator <= (const String s1, const String s2) { return !(s1 > s2); } } </pre>	<pre> 构造了一个String对象 构造了一个String对象 s1 = abce s2 = abde 复制构造了一个String对象 复制构造了一个String对象 复制构造了一个String对象 复制构造了一个String对象 析构了一个String对象 析构了一个String对象 复制构造了一个String对象 复制构造了一个String对象 析构了一个String对象 析构了一个String对象 复制构造了一个String对象 复制构造了一个String对象 析构了一个String对象 析构了一个String对象 s1 < s2 复制构造了一个String对象 复制构造了一个String对象 复制构造了一个String对象 复制构造了一个String对象 析构了一个String对象 析构了一个String对象 析构了一个String对象 析构了一个String对象 析构了一个String对象 析构了一个String对象 s1 != s2 复制构造了一个String对象 复制构造了一个String对象 复制构造了一个String对象 复制构造了一个String对象 析构了一个String对象 析构了一个String对象 析构了一个String对象 析构了一个String对象 s1 <= s2 析构了一个String对象 析构了一个String对象 s1 = fghj s2 = fejk 复制构造了一个String对象 复制构造了一个String对象 复制构造了一个String对象 复制构造了一个String对象 析构了一个String对象 析构了一个String对象 析构了一个String对象 析构了一个String对象 s1 > s2 复制构造了一个String对象 复制构造了一个String对象 析构了一个String对象 析构了一个String对象 </pre>

```

String& operator = (const String s2) {
    if (this != &s2) {
        delete[] str;
        this->str = new char[strlen(s2.str)
+ 1];
        strcpy(str, s2.str);
    }
    return *this;
};

void cmp(String s1, const String s2) {
    if (s1 > s2) cout << "s1 > s2" << endl;
    if (s1 == s2) cout << "s1 == s2" << endl;
    if (s1 < s2) cout << "s1 < s2" << endl;
    if (s1 >= s2) cout << "s1 >= s2" << endl;
    if (s1 != s2) cout << "s1 != s2" << endl;
    if (s1 <= s2) cout << "s1 <= s2" << endl;
}

int main() {
    String s1("abce"), s2("abde");
    cout << "s1 = "; s1.Print();
    cout << "s2 = "; s2.Print();
    cmp(s1, s2); cout << endl;
    s1.Set("fghj"); s2.Set("fejk");
    cout << "s1 = "; s1.Print();
    cout << "s2 = "; s2.Print();
    cmp(s1, s2); cout << endl;
    s1.Set("pubg"); s2 = s1;
    cout << "s1 = "; s1.Print();
    cout << "s2 = "; s2.Print();
    cmp(s1, s2);
    return 0;
}

```

复制构造了一个 String 对象
 复制构造了一个 String 对象
 析构了一个 String 对象
 析构了一个 String 对象
 复制构造了一个 String 对象
 复制构造了一个 String 对象
 复制构造了一个 String 对象
 复制构造了一个 String 对象
 析构了一个 String 对象
 析构了一个 String 对象
 析构了一个 String 对象
 析构了一个 String 对象
 s1 >= s2
 复制构造了一个 String 对象
 复制构造了一个 String 对象
 复制构造了一个 String 对象
 复制构造了一个 String 对象
 析构了一个 String 对象
 析构了一个 String 对象
 析构了一个 String 对象
 析构了一个 String 对象
 析构了一个 String 对象
 析构了一个 String 对象
 s1 != s2
 复制构造了一个 String 对象
 复制构造了一个 String 对象
 复制构造了一个 String 对象
 复制构造了一个 String 对象
 析构了一个 String 对象
 析构了一个 String 对象
 析构了一个 String 对象
 析构了一个 String 对象
 析构了一个 String 对象
 析构了一个 String 对象
 复制构造了一个 String 对象
 析构了一个 String 对象
 s1 = pubg
 s2 = pubg
 复制构造了一个 String 对象
 复制构造了一个 String 对象
 复制构造了一个 String 对象
 复制构造了一个 String 对象
 析构了一个 String 对象
 析构了一个 String 对象
 复制构造了一个 String 对象
 复制构造了一个 String 对象
 析构了一个 String 对象
 析构了一个 String 对象
 复制构造了一个 String 对象
 复制构造了一个 String 对象
 复制构造了一个 String 对象
 复制构造了一个 String 对象
 析构了一个 String 对象
 析构了一个 String 对象
 析构了一个 String 对象
 析构了一个 String 对象
 s1 == s2
 复制构造了一个 String 对象
 复制构造了一个 String 对象
 析构了一个 String 对象
 析构了一个 String 对象
 复制构造了一个 String 对象
 复制构造了一个 String 对象
 复制构造了一个 String 对象
 复制构造了一个 String 对象
 析构了一个 String 对象
 析构了一个 String 对象
 析构了一个 String 对象
 析构了一个 String 对象
 s1 >= s2
 复制构造了一个 String 对象
 复制构造了一个 String 对象
 复制构造了一个 String 对象

	复制构造了一个 String 对象 析构了一个 String 对象 析构了一个 String 对象 析构了一个 String 对象 析构了一个 String 对象 复制构造了一个 String 对象 复制构造了一个 String 对象 复制构造了一个 String 对象 复制构造了一个 String 对象 析构了一个 String 对象 析构了一个 String 对象 析构了一个 String 对象 析构了一个 String 对象 s1 <= s2 析构了一个 String 对象 析构了一个 String 对象 析构了一个 String 对象 析构了一个 String 对象
--	---

C++ code	Result
重载为成员函数	构造了一个 String 对象 构造了一个 String 对象
<pre> #include<iostream> #include<cstring> using namespace std; #pragma warning(disable:4996) class String { private: char* str; public: String(const char* s) { str = new char[strlen(s) + 1]; strcpy(str, s); cout << "构造了一个String对象" << endl; } String() { str = new char[2]; strcpy(str, "\\0"); cout << "构造了一个String对象" << endl; } String(const String& STR) { str = new char[strlen(STR.str) + 1]; strcpy(str, STR.str); cout << "复制构造了一个String对象" << endl; } ~String() { delete[]str; cout << "析构了一个 String对象" << endl; } void Set(const char* s) { delete[]this->str; this->str = new char[strlen(s) + 1]; strcpy(str, s); } void Print() { cout << str << endl; } bool operator == (const String& s2) { return !strcmp(str, s2.str); } bool operator > (const String& s2) { if (strcmp(str, s2.str) == 1) return true; else return false; } bool operator < (const String& s2) { if (strcmp(str, s2.str) == -1) return true; else return false; } bool operator != (const String& s2) { return !(*this == s2); } bool operator >= (const String& s2) { return !(*this < s2); } bool operator <= (const String& s2) { return !(*this > s2); } String& operator = (const String& s2) { if (this != &s2) { delete[] str; this->str = new char[strlen(s2.str) </pre>	<pre> s1 = abce s2 = abde s1 < s2 s1 != s2 s1 <= s2 s1 = fghj s2 = fejk s1 > s2 s1 >= s2 s1 != s2 s1 = pubg s2 = pubg s1 == s2 s1 >= s2 s1 <= s2 析构了一个 String 对象 析构了一个 String 对象 </pre>

```

+ 1];
        strcpy(str, s2.str);
    }
    return *this;
}

};

void cmp(String& s1, const String& s2) {
    if (s1 > s2) cout << "s1 > s2" << endl;
    if (s1 == s2) cout << "s1 == s2" << endl;
    if (s1 < s2) cout << "s1 < s2" << endl;
    if (s1 >= s2) cout << "s1 >= s2" << endl;
    if (s1 != s2) cout << "s1 != s2" << endl;
    if (s1 <= s2) cout << "s1 <= s2" << endl;
}

int main() {
    String s1("abce"), s2("abde");
    cout << "s1 = "; s1.Print();
    cout << "s2 = "; s2.Print();
    cmp(s1, s2);
    s1.Set("fghj"); s2.Set("fejk");
    cout << "s1 = "; s1.Print();
    cout << "s2 = "; s2.Print();
    cmp(s1, s2);
    s1.Set("pubg"); s2 = s1;
    cout << "s1 = "; s1.Print();
    cout << "s2 = "; s2.Print();
    cmp(s1, s2);
    return 0;
}

```

重载为友元函数

```

#include<iostream>
#include<cstring>
using namespace std;

#pragma warning(disable:4996)

class String {
private:
    char* str;
public:
    String(const char* s) {
        str = new char[strlen(s) + 1];
        strcpy(str, s);
        cout << "构造了一个String对象" << endl;
    }
    String() {
        str = new char[2]; strcpy(str, "\0");
        cout << "构造了一个String对象" << endl;
    }
    String(const String& STR) {
        str = new char[strlen(STR.str) + 1];
        strcpy(str, STR.str);
        cout << "复制构造了一个String对象" <<
endl;
    }
    ~String() { delete[] str; cout << "析构了一个
String对象" << endl; }
    void Set(const char* s) {
        delete[] this->str;
        this->str = new char[strlen(s) + 1];
        strcpy(str, s);
    }
}

```

```

    void Print() { cout << str << endl; }
    friend bool operator == (const String& s1,
const String& s2) {
        return !strcmp(s1.str, s2.str);
    }
    friend bool operator > (const String&
s1, const String& s2) {
        if (strcmp(s1.str, s2.str) == 1) return
true;
        else return false;
    }
    friend bool operator < (const String& s1,
const String& s2) {
        if (strcmp(s1.str, s2.str) == -1) return
true;
        else return false;
    }
    friend bool operator != (const String& s1,
const String& s2) {
        return !(s1 == s2);
    }
    friend bool operator >= (const String& s1,
const String& s2) {
        return !(s1 < s2);
    }
    friend bool operator <= (const String& s1,
const String& s2) {
        return !(s1 > s2);
    }
    String& operator = (const String& s2) {
        if (this != &s2) {
            delete[] str;
            this->str = new char[strlen(s2.str)
+ 1];
            strcpy(str, s2.str);
        }
        return *this;
    }
};

void cmp(String& s1, const String& s2) {
    if (s1 > s2) cout << "s1 > s2" << endl;
    if (s1 == s2) cout << "s1 == s2" << endl;
    if (s1 < s2) cout << "s1 < s2" << endl;
    if (s1 >= s2) cout << "s1 >= s2" << endl;
    if (s1 != s2) cout << "s1 != s2" << endl;
    if (s1 <= s2) cout << "s1 <= s2" << endl;
}

int main() {
    String s1("abce"), s2("abde");
    cout << "s1 = "; s1.Print();
    cout << "s2 = "; s2.Print();
    cmp(s1, s2); cout << endl;
    s1.Set("fghj"); s2.Set("fejk");
    cout << "s1 = "; s1.Print();
    cout << "s2 = "; s2.Print();
    cmp(s1, s2); cout << endl;
    s1.Set("pubg"); s2 = s1;
    cout << "s1 = "; s1.Print();
    cout << "s2 = "; s2.Print();
    cmp(s1, s2);
    return 0;
}

```

把形参改为引用方式后，程序可大大减少调用复制构造函数的次数，同时减少调用析构函数的次数，甚至可以完全不调用。这样的改良方式可以大大减少不必要的执行步骤，提高程序执行的效率，